

0011 03.Module Scope...Local Module 3 :IIFE Module

Tuesday, 27 June 2023 6:26 PM

Create two files:-

e.g. 1

batman.js

```
const superhero = "batman";  
console.log(superhero);
```

superman.js

```
const superhero = "superman";  
console.log(superhero);
```

index.js

```
require("./batman");  
require("./superman");
```

```
%node index  
batman  
superman
```

- We are importing two modules, which containing exact same constant variable - **superhero**
- No conflict or error $\because$ each module in node.js has its own scope.

e.g. 2

iife - file pattern

iife.js

```
(function() {  
  const superhero = "Batman";  
  console.log(superhero);  
})(); // one separate module.
```

```
(function() {  
  const superhero = "Superman";  
  console.log(superhero);  
})(); // 2nd separate module.
```

```
%node iife  
Batman  
Superman  
%
```

(function(){})();

→ Two combine process

1. fⁿ definition

function() {}

2. immediate fⁿ. call

(function(){})();

iife() pattern

- This is because with iife's each function gets its own private scope.
- Node.js as well uses same iife pattern.

*** IIFE** : Immediately Invoked Function Expression in Node.js

```
(function() {  
  // module code  
})
```

- Before module's code is executed, Node.js will wrap it with a **function wrapper** that provides **module scope**.
- This saves us from having to worry about conflicting variables or functions.
- There is proper encapsulation & reusability is unaffected.

*** Summary:-**

- Each loaded module in Node.js is wrapped with an IIFE, that provides private scoping of code.
- IIFE allows you to repeat variable or function names without any conflict.